

位置入口：

位置 A：通关结算页（最推荐，第一优先级）

通关后弹出结算卡，在「再来一局」按钮右边放一个按钮：

[分享战绩]

用户刚通关，情绪好，分享意愿最强。而且战绩卡有具体内容（分数+猫咪），好友收到后有点进来的动力，形成真实的拉新。

位置 B：首页每日任务列表（已有，保留）

你现在已有这个位置，保留即可。适合习惯性完成任务的玩家，但转化率不如结算页高，作为补充入口。

位置 C：历史最高分打破时（彩蛋触发）

打破历史最高分时的高光弹窗里，增加一行：

新纪录！2,480 分猫咪都为你鼓掌！[分享] ← 情绪最高点，转化率最高

这是锦上添花，不占固定位置，在特定时刻出现，用户感觉是自然的庆祝行为而不是任务。

位置 D：家园页猫咪卡片长按（低干扰）

长按猫咪卡片弹出操作菜单，加一项「推荐这只猫给朋友」。适合对猫咪有感情的玩家，触发率低但质量高，被推荐者进来往往更容易留存。

优先级建议

P0（上线时必须有） → 结算页次级按钮（转化率最高） → 首页每日任务（已有保留） P1（第一次迭代）
→ 历史最高分弹窗内分享按钮 P2（后续版本） → 家园猫咪长按菜单

二、完整功能流程

2.1 核心流程（用一句话描述）

核心逻辑 玩家 A 分享今日战绩卡片 → 好友 B 点击进入游戏 → 好友 B 完成第一局消消乐 → A 和 B 各得鱼干

×30。每个玩家每日最多触发 3 次奖励（即最多推荐 3 位新好友）。

2.2 详细流程拆解

流程共分五个阶段：

阶段	触发节点	执行内容	涉及角色
阶段 1： 生成分享	玩家点击「推荐游戏」按钮	后端生成专属邀请码（inviteCode），绑定玩家 ID 和时间戳，有效期 24 小时	玩家 A + 后端
阶段 2： 分享卡片	系统弹出微信分享面板	分享卡片展示今日战绩（得分+消除数），携带 inviteCode 参数，好友点击后进入游戏首页	玩家 A + 微信
阶段 3： 新用户进入	好友 B 通过分享卡片首次进入游戏	前端读取 URL 中的 inviteCode，存入本地缓存；校验 B 是否为新用户（从未打开过游戏）	好友 B + 前端
阶段 4： 完成第一局	好友 B 完成第一局消乐（通关或失败均算）	前端触发奖励校验请求，后端验证全部条件，通过后发放双方奖励	好友 B + 后端
阶段 5： 奖励到账	奖励发放成功	A 和 B 各收到鱼干×30，游戏内 Toast 提示，任务列表更新完成状态	玩家 A + 好友 B

三、详细业务规则

3.1 奖励规则

规则项	内容	说明
奖励类型	鱼干×30（双方各得）	选鱼干不选铃铛，降低合规风险，后续版本可升级
每日上限	每位玩家每日最多获得 3 次推荐奖励	即每日最多+90 条鱼干，防止刷量
好友限定	好友 B 必须是首次打开游戏的新用户	老用户通过分享进入不触发奖励
完成条件	好友 B 完成第一局消消乐（通关或失败均算）	降低门槛，让被推荐者体验友好
邀请码有效期	24 小时	过期后需重新生成分享
重复防护	同一好友 B 只能被记录一次	B 被多人分享，只有最早进入的一条 inviteCode 有效

3.2 边界情况处理

边界情况	处理方式
好友 B 是老用户（注册过但未登录）	不触发奖励，静默处理，不提示错误
好友 B 完成游戏时 A 的当日次数已达上限	B 完成游戏，B 不得奖励（A 已满），仅 B 得奖励×30
inviteCode 已过期（超过 24 小时）	B 进入游戏时存储 inviteCode 但标记为过期，B 完成游戏后不发放奖励
B 通过分享进入但未完成游戏就关闭	inviteCode 记录保留，B 下次打开游戏内仍可完成第一局触发奖励（7 日内有效）
网络异常导致奖励发放失败	奖励进入待发放队列，5 分钟内重试 3 次，仍失败则人工处理
A 被封号或注销	奖励不发放给 A，B 的奖励正常发放

四、前端实现

4.1 每日任务入口（首页）

在首页每日任务列表中展示推荐任务，样式与其他每日任务保持一致：

元素	内容	说明
任务图标		推荐分享类图标
任务标题	「把游戏推荐给朋友」	不出现「分享」「邀请」「助力」等敏感词
任务描述	「朋友体验游戏后，双方各得鱼干×30」	明确说明奖励规则，合规要求
进度显示	「今日已完成 X/3 次」	X 为当日已触发奖励次数，上限 3
按钮文案	「去推荐」	简洁，不诱导
完成状态	按钮置灰，显示「今日已完成」	达到 3 次上限后不可再点

4.2 点击「去推荐」按钮流程

按钮点击后执行以下步骤：

1. 调用后端接口生成 inviteCode（POST /share/generate）
2. 拿到 inviteCode 后调用微信分享 API（wx.shareAppMessage）
3. 分享完成回调：前端记录本次分享行为（用于展示「已分享」状态）

```
// pages/home/home.js

async onShareTap() {
  // 检查当日次数
  if (this.data.todayShareCount >= 3) {
    wx.showToast({ title: "今日推荐次数已用完", icon: "none" });
    return;
  }

  try {
    // 1. 生成邀请码
    const res = await request("POST", "/share/generate");
    const { inviteCode } = res.data;

    // 2. 触发微信分享（用户主动触发，符合微信规范）
    wx.shareAppMessage({
      title: `我在喵喵乐消消打了${this.data.todayScore}分，来挑战我吧`,
      path: `/pages/index/index?inviteCode=${inviteCode}`,
      imageUrl: "/assets/share/share-cover.png",
    });
  } catch (e) {}
}
```

```
wx.showToast({ title: "分享失败, 请重试", icon: "none" });
}
},
```

注意 `wx.shareAppMessage` 只能在用户点击事件中直接调用，不能在异步回调中调用，否则微信会拦截。需要先拿到 `inviteCode` 再调用分享，因此后端接口响应必须在 300ms 内完成。

4.3 好友 B 进入游戏时的处理

```
// app.js onLaunch

onLaunch(options) {
  // 读取分享链接中的 inviteCode
  const inviteCode = options.query?.inviteCode;
  if (inviteCode) {
    // 存入本地缓存，等待完成第一局后使用
    wx.setStorageSync("pendingInviteCode", inviteCode);
    wx.setStorageSync("inviteCodeTime", Date.now());
  }
},
```

```
// pages/game/game.js 关卡结算时触发

async onLevelEnd(result) {
  // 正常结算逻辑...
  await this.settleFishCount(result);

  // 检查是否有待处理的邀请码
  const inviteCode = wx.getStorageSync("pendingInviteCode");
  const inviteTime = wx.getStorageSync("inviteCodeTime");

  if (inviteCode) {
    // 检查邀请码是否在 7 日内（兼容关闭游戏再次进入的场景）
    const isValid = Date.now() - inviteTime < 7 * 24 * 60 * 60 * 1000;
    if (isValid) {
      await this.claimInviteReward(inviteCode);
    }
    // 无论成功失败，清除本地缓存，避免重复触发
    wx.removeStorageSync("pendingInviteCode");
    wx.removeStorageSync("inviteCodeTime");
  }
},

async claimInviteReward(inviteCode) {
  try {
    const res = await request("POST", "/invite/complete", { inviteCode });
    if (res.data.success) {
      // 显示奖励 Toast
      wx.showToast({
        title: " 好友推荐成功! +30 鱼干",
        icon: "none",
        duration: 2500,
      });
      // 刷新鱼干数量
    }
  }
}
```

```
        this.refreshFishCount();
    }
} catch(e) {
    // 静默失败，不影响正常游戏流程
    console.error("邀请奖励发放失败:", e);
}
},
```

4.4 分享卡片设计规范

元素	内容	规范
卡片标题	「我在喵呜乐消消打了 X 分，来挑战我吧」	动态插入今日最高分，有内容价值
卡片图片	展示今日出战猫咪+分数的战绩截图	尺寸 500×400px，PNG 格式
跳转路径	/pages/index/index?inviteCode=xxxx	携带邀请码参数
卡片描述	「养好猫，闯难关」	游戏口号，不出现奖励相关文案



卡片图参考：/Users/wangguangzheng/Desktop/喵了个咪/cat/欢迎图.jpg

五、后端实现

5.1 数据库新增字段

表名	新增字段	类型	说明
users	invite_code	VARCHAR(32)	当日有效邀请码，每次分享更新
users	invite_code_expire	TIMESTAMP	邀请码过期时间
users	today_invite_count	INT DEFAULT 0	今日已成功推荐次数（每日 0 点重置）
users	today_invite_reset	DATE	最后重置日期（用于判断是否需要重置）
invite_records（新建表）	id	BIGINT PK	主键
invite_records	inviter_id	BIGINT	邀请人（玩家 A）的用户 ID
invite_records	invitee_id	BIGINT	被邀请人（好友 B）的用户 ID
invite_records	invite_code	VARCHAR(32)	使用的邀请码
invite_records	status	ENUM	pending=待完成 / completed=已完成 / expired=已过期
invite_records	created_at	TIMESTAMP	记录创建时间
invite_records	completed_at	TIMESTAMP	完成时间（好友完成第一局的时间）

5.2 接口一：生成邀请码

POST /share/generate
Headers: Authorization: Bearer {token}
Body: 无
处理逻辑:
1. 验证用户登录态
2. 检查今日推荐次数 (today_invite_count >= 3 则返回错误)
3. 生成邀请码: MD5(userId + timestamp + 随机盐) 取前 16 位
4. 更新 users 表: invite_code = 新码, invite_code_expire = now()+24h
5. 在 invite_records 插入一条 status=pending 的记录
返回:
200 { success: true, inviteCode: "abc123def456xyz" }
400 { success: false, error: "today_limit_reached" } // 当日次数已满

5.3 接口二：完成邀请奖励

POST /invite/complete
Headers: Authorization: Bearer {token} // 好友 B 的 token

```
Body: { inviteCode: "abc123def456xyz" }
```

处理逻辑（事务内执行，失败全部回滚）：

1. 验证好友B的登录态，获取 inviteeId
2. 查询 invite_records，找到对应 inviteCode 的记录
 - 不存在：返回 404
 - status != pending: 返回 400（已完成或已过期）
 - invite_code_expire < now(): 标记为 expired，返回 400
3. 验证好友B是否为新用户（users.created_at 距今 < 7天 且 首次登录）
 - 是老用户：返回 400 { error: "not_new_user" }
4. 验证好友B是否已被其他邀请码记录（invite_records 有B的 completed 记录）
 - 已存在：返回 400 { error: "already_invited" }
5. 获取邀请人A的userId（从 invite_records.inviter_id）
6. 检查A的 today_invite_count < 3（A未达上限才给A发奖励）
7. 发放奖励：
 - 给B: UPDATE users SET fish_count = fish_count+30 WHERE id=inviteeId
 - 给A（如未达上限）: UPDATE users SET fish_count = fish_count+30,
today_invite_count = today_invite_count+1
WHERE id=inviterId
8. 更新 invite_records: status=completed, completed_at=now(), invitee_id=inviteeId

返回：

```
200 { success: true,  
      inviterRewarded: true/false, // A 是否得到奖励  
      inviteeReward: 30,           // B 获得的鱼干数  
      inviterReward: 30 }         // A 获得的鱼干数（0 或 30）  
400 { success: false, error: "..." }
```

5.4 每日次数重置（定时任务）

```
// 每日北京时间 00:00 执行  
UPDATE users  
  SET today_invite_count = 0,  
      today_invite_reset = CURDATE()  
 WHERE today_invite_reset < CURDATE();  
  
// 同时清理过期的 invite_records  
UPDATE invite_records  
  SET status = "expired"  
 WHERE status = "pending"  
    AND created_at < NOW() - INTERVAL 7 DAY;
```


六、测试用例

6.1 正常流程测试

编号	测试场景	操作步骤	预期结果
TC01	完整正常流程	1.A 点击推荐按钮 2.生成 inviteCode 3.B 通过分享进入 4.B 完成第一局	A 和 B 各收到+30 鱼干，Toast 提示，任务列表显示「今日 1/3」
TC02	A 连续推荐 3 人	A 分别让 B1/B2/B3 完成第一局	A 得 9 条鱼干，任务显示「今日 3/3 已完成」，按钮置灰
TC03	B 关闭游戏后再回来	B 通过分享进入游戏，未完成游戏关闭，7 日内重新打开完成第一局	奖励正常触发，本地缓存 inviteCode 7 日内有效

6.2 边界情况测试

编号	测试场景	预期结果
TC04	A 已达今日 3 次上限再分享	接口返回 today_limit_reached，前端提示「今日推荐次数已用完」
TC05	老用户 B 通过分享进入	后端返回 not_new_user，前端静默处理，B 游戏正常进行，不显示错误
TC06	B 已被别人邀请过	返回 already_invited，B 的奖励不重复发放
TC07	邀请码过期后 B 完成游戏	返回 expired，双方均不发放奖励
TC08	B 完成游戏时 A 已达上限	B 正常获得+30 鱼干，A 不获得奖励，inviter_rewarded 返回 false
TC09	接口超时或异常	前端静默失败，不影响游戏结算，用户不感知错误
TC10	快速重复点击推荐按钮	后端幂等处理，同一 inviterCode 不重复写入记录